PHASE 3 – PROJECT DESIGN

AI FAQ ASSISTANT API

AI-Powered FAQ and Customer Support Automation

---

1. INTRODUCTION

Project Design defines the structure and working flow of the **AI FAQ Assistant API** based on the requirements identified in Phase 2.

The system is designed as a modular RESTful backend application using **Node.js and Express.js**. MongoDB is used for data storage through Mongoose, while Google Gemini is integrated for AI-based answer and FAQ generation.

The design separates the application into different components such as routes, controllers, middleware, models, services, configuration, and utility functions.

---

2. SYSTEM ARCHITECTURE

The AI FAQ Assistant follows a layered **MVC-based backend architecture**.

Basic Architecture

Client / Postman

↓

Express.js API

↓

Routes

↓

Middleware

↓

Controllers

↓

Models / Services

↓

MongoDB / Google Gemini

Main Components

Component	Responsibility
Client	Sends API requests
Express.js	Handles HTTP requests
Routes	Direct requests to appropriate controllers
Middleware	Authentication, validation and error handling
Controllers	Process application operations
Models	Define MongoDB data structures
Services	Handle Google Gemini operations
MongoDB	Stores users and FAQs
Google Gemini	Generates AI responses

The project architecture also describes an Express gateway, middleware, controller layer, database layer, and external AI service integration.

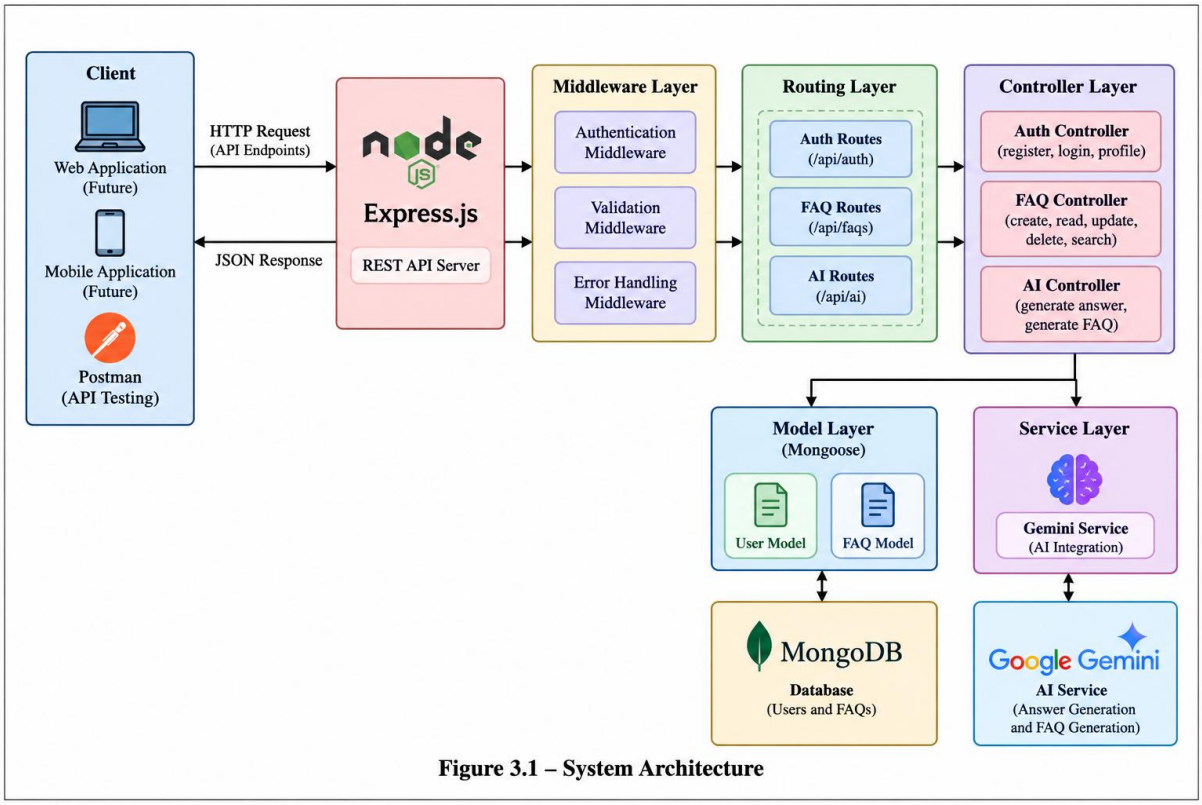


Figure 3.1 – System Architecture

Figure 3.1 – AI FAQ Assistant System Architecture

3. MVC DESIGN

The project uses the **Model-View-Controller (MVC)** approach.

Since this project is a backend REST API, the View is represented by the API response rather than a traditional HTML view.

Model

The Model layer contains the MongoDB schemas.

Current models:

- User
- FAQ

Controller

The Controller layer processes requests and performs application operations.

Current controllers:

- Authentication Controller
- FAQ Controller
- AI Controller

Route

Routes define the API endpoints and connect requests to their respective controllers.

Middleware

Middleware performs common operations before or after controller execution.

Examples:

- Authentication
- Validation
- Error handling

Service

The AI service manages communication with Google Gemini.

The supplied project documentation describes the Model layer as the database structure, the Controller layer as the request-processing component, and the routing layer as the interface between requests and controllers.

---

4. PROJECT MODULE DESIGN

The system is divided into the following major modules.

Module	Main Responsibility
Authentication	Registration, login and profile
FAQ Management	FAQ CRUD operations
FAQ Search	Search existing FAQs
AI Assistant	AI answers and FAQ generation
Database	Store users and FAQs
Security	Authentication and validation
Error Handling	Manage application errors

Authentication Module

Handles:

- User registration
- User login
- Password hashing
- JWT generation
- Profile access

FAQ Module

Handles:

- Create FAQ
- Retrieve FAQs
- Update FAQ
- Delete FAQ
- Search FAQ

AI Module

Handles:

- AI question answering
- AI FAQ generation

---

5. DATABASE DESIGN

MongoDB is used as the database for the project.

Mongoose is used to define and manage the database schemas.

The current implementation contains two primary models:

1. **User**
2. **FAQ**

5.1 User Model

Field	Type	Description
name	String	User name
email	String	User email
password	String	Hashed password
createdAt	Date	Account creation time
updatedAt	Date	Last update time

5.2 FAQ Model

Field	Type	Description
question	String	FAQ question
answer	String	FAQ answer
category	String	FAQ category
createdBy	ObjectId	FAQ creator
createdAt	Date	FAQ creation time
updatedAt	Date	Last update time

6. DATABASE RELATIONSHIP DESIGN

The main relationship between the database entities is:

User → FAQ

One user can create multiple FAQs.

USER (1) ———————— (M) FAQ

The created By field in the FAQ model stores a reference to the user who created the FAQ.

This relationship is also used during update and delete operations to verify that the authenticated user owns the FAQ.

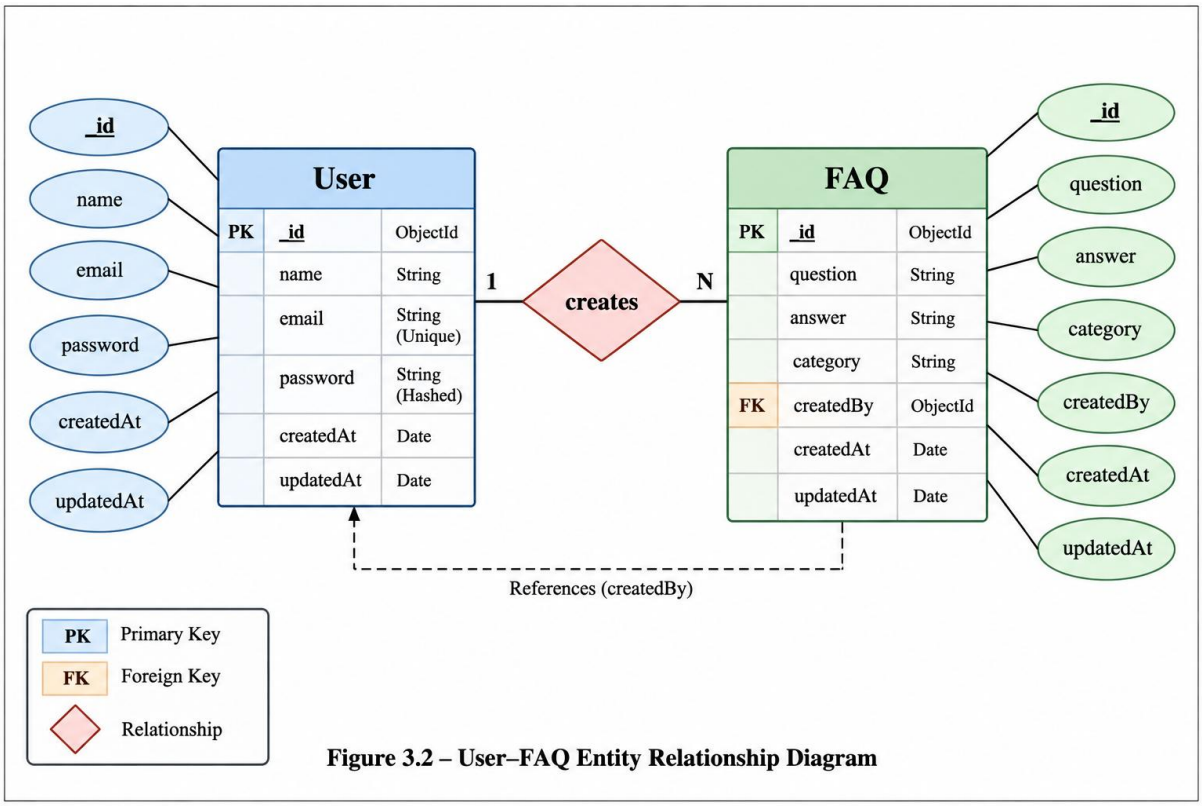


Figure 3.2 – User and FAQ Entity Relationship Diagram

Simple ER Representation

USER

- User ID
- Name
- Email
- Password

↓

creates

↓

FAQ

- FAQ ID
- Question
- Answer
- Category
- Created By

The source project similarly defines the User-to-FAQ relationship as one-to-many, with a registered user able to create multiple FAQ records.

---

7. API DESIGN

The application provides RESTful endpoints for authentication, FAQ management, search, and AI operations.

Authentication

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user
GET	/api/auth/profile	Retrieve profile

FAQ

Method	Endpoint	Purpose
POST	/api/faqs	Create FAQ
GET	/api/faqs	Get all FAQs
GET	/api/faqs/:id	Get FAQ by ID
PUT	/api/faqs/:id	Update FAQ
DELETE	/api/faqs/:id	Delete FAQ
GET	/api/faqs/search?q=	Search FAQs

AI

Method	Endpoint	Purpose
POST	/api/ai/answer	Generate AI answer
POST	/api/ai/generate-faq	Generate FAQ using AI

The supplied project documentation demonstrates the same basic REST endpoint design for authentication, FAQ creation/search, and AI FAQ generation.

---

8. SECURITY AND REQUEST FLOW DESIGN

Protected operations use JWT authentication.

Authentication Flow

User

↓

Login Request

↓

Authentication Controller

↓

Verify Email and Password

↓

Generate JWT

↓

Return Token

↓

Client sends Bearer Token

↓

Authentication Middleware

↓

Verify Token

↓

Protected Controller

↓

Response

FAQ Ownership Flow

Authenticated Request

↓

JWT Verification

↓

Identify User

↓

Find FAQ

↓

Check createdBy

↓

Allow / Reject Operation

This design prevents an authenticated user from modifying or deleting another user's FAQ.

The supplied system architecture identifies authentication middleware as the component responsible for extracting authorization information and validating token identity.

---

9. AI MODULE DESIGN

The AI module connects the backend application with **Google Gemini**.

Two major AI operations are designed.

9.1 AI Answer Generation

Flow

User Question

↓

AI Route

↓

AI Controller

↓

Gemini Service

↓

Google Gemini

↓

Generated Answer

↓

API Response

9.2 AI FAQ Generation

Flow

Topic

↓

AI Route

↓

AI Controller

↓

Gemini Service

↓

Google Gemini

↓

Generated Question + Answer

↓

API Response

The project documentation describes Google Gemini integration for automated question-answering and FAQ generation.

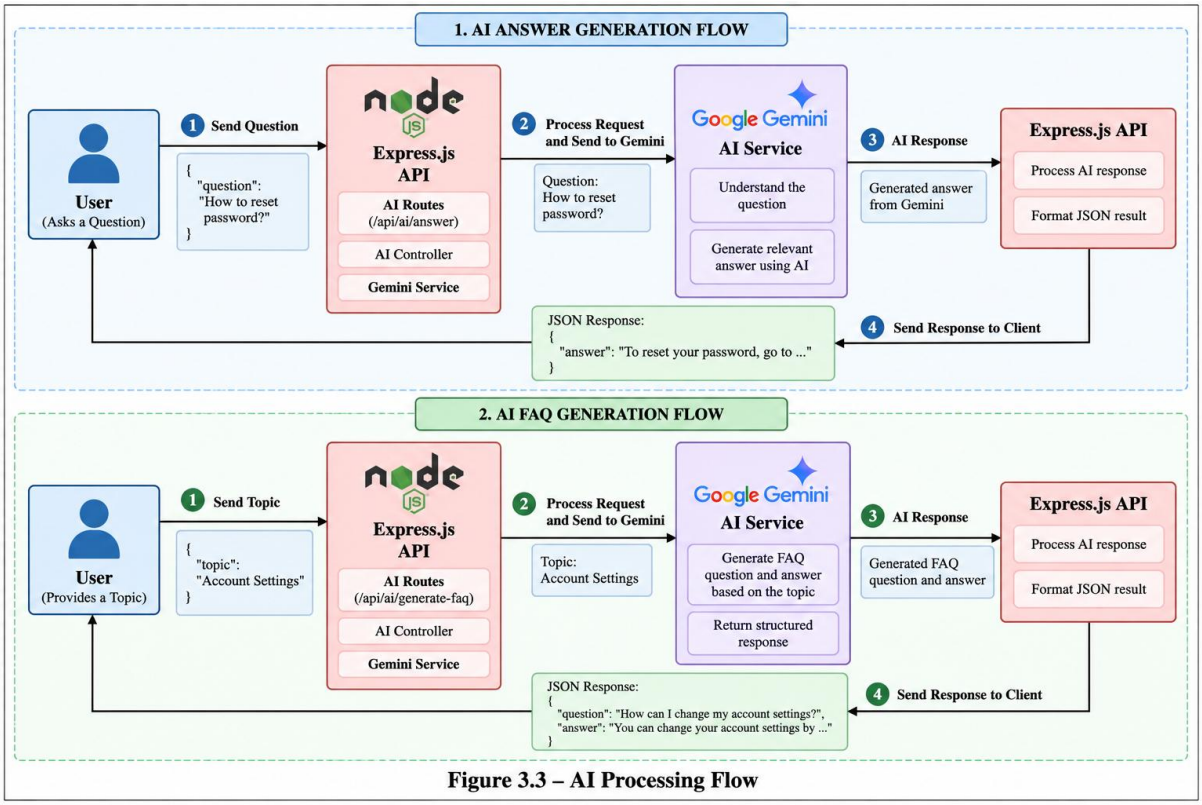


Figure 3.3 – AI Processing Flow

Figure 3.3 – AI Processing Flow

---

10. PROJECT FOLDER DESIGN AND DESIGN SUMMARY

The project follows a modular folder structure.

Folder/File	Purpose
config/	Database configuration
controllers/	Request processing
middleware/	Authentication, validation and errors

Folder/File	Purpose
models/	MongoDB schemas
routes/	API endpoints
services/	Gemini AI integration
utils/	Helper functions
app.js	Express application configuration
server.js	Server startup
test_api.js	API testing
package.json	Project dependencies and scripts

Overall Request Flow

Client

→ Route

→ Middleware

→ Controller

→ Model / Service

→ MongoDB / Gemini

→ Controller

→ JSON Response

The design provides a clear separation between API routing, business logic, database operations, authentication, and AI processing.

This architecture also allows additional functionality to be added in future without significantly changing the existing structure.
